

모의 양자화와 정수 실행의 격차를 연산자 단위로 측정하기

Measuring the Fake-Quantization-to-Integer Gap,
Operator by Operator

INT8 NPU 배포를 위한 비트 정확 정수 엔진과 그 위에서의 측정

윤기혁

2026년 9월

개인 연구 원고 · 학위 제출물 및 게재 논문 아님

저장소 <https://github.com/sokldjs554/npuloop>

초 록

INT8 배포를 준비할 때 모의(fake) 양자화의 부동소수점 그래프와 실제 정수 연산 경로가 얼마나 일치하는지 확인해야 한다. 이 원고는 동일한 양자화 파라미터를 사용하는 두 실행 경로를 정확도와 정수 코드의 두 관점에서 비교한다. 정수 엔진은 TensorFlow Lite 참조 커널의 합성곱·평균 풀링·완전 연결 계층을 포함한 시험 모델에서, 같은 스케일과 영점을 사용한 1,000장 입력의 비교 대상 모든 텐서와 비트 단위로 일치했다. 이 검증은 다른 연산자나 실제 NPU 하드웨어 전체의 일치를 보장하지 않는다.

신경망 7종과 양자화 방식 2종을 평가했다. 분류 모델 6종의 12개 비교에서 관측된 (정수 – 모의) top-1 차이의 절댓값은 최대 0.20 %p였고, 각 95% 신뢰구간은 모두 0을 포함했다. 이는 동등성의 입증이 아니다. 같은 비교에서 출력 코드의 28.5–72.7%는 달랐다. 품질과 출력 코드 불일치는 전체 테스트셋에서, 교사 강제 방식의 연산자별 국소 불일치는 고정 표본 배치에서 측정했다.

연산자 종류별로 노드별 국소 불일치율을 단순평균하면 LayerNorm 이외의 평균은 모두 1% 미만이었다. 다만 평균 풀링의 개별 노드 최댓값은 2.697%였다. 관측된 비영 국소 코드 차이는 모두 ± 1 LSB였다. LayerNorm만 정수 연산으로 에뮬레이션하면 국소 불일치는 24.77%에서 0.00%가 되지만 출력 코드 불일치는 72.7%에서 65.7%로만 줄었다. 이는 LayerNorm 단독 교체의 한계이며, 모든 연산자의 누적 교체 가능성을 배제하지 않는다.

초해상 모델에서도 두 실행의 평균 PSNR 차이는 최대 약 6.4×10^{-4} dB인 반면, 출력 픽셀 코드의 6.3–18.2%가 달랐다. 폭과 과제를 고정한 채 합성곱을 3개에서 9개로 늘리면 이 코드 불일치는 69.2–73.0%로 올라 분류기들의 구간 안으로 들어오는데, PSNR 차이는 여전히 2.9×10^{-3} dB 이하였다. 전파 불일치의 크기를 정하는 것은 과제가 아니라 깊이다. 재양자화의 반올림을 제거한 truncate-floor는 3개 모델 × 3개 학습 반복 × 2개 반올림 모드의 18개 비교 모두에서 기준보다 0.90–3.37 %p 낮았다. 또한 초해상 입력으로 드러난 입력 양자화기의 정밀도 결함을 진단·수정한 과정을 기록한다. 결과 파일에 근거한 표 재생성과 기존 실험의 독립 재실행은 구분하며, 학습 원본 체크포인트의 미공개 한계를 명시한다.

주제어: 양자화, 정수 연산, 임베디드 추론, 신경망 처리 장치, 재현성

차 례

초록	i
1 서론	1
2 배경	2
2.1 INT8 정수 데이터패스	2
2.2 모의 양자화와 정수 실행	2
2.3 관련 연구	3
3 동기: 격차는 어디에 있는가	4
3.1 연산자별 불일치 분류	4
3.2 무엇이 문제인가	5
4 방법	6
4.1 그래프 IR과 정수 그래프 export	6
4.2 정수 엔진과 외부 검증	6
4.3 국소 불일치와 전파 불일치	6
4.4 LayerNorm 정수 에뮬레이션	7
4.5 통계 처리	7
5 성능 평가	8
5.1 실험 설정	8
5.2 정확도와 텐서의 분리	9
5.3 LayerNorm 어블레이션	9
5.4 출력 텐서가 결과물인 과제	11
5.5 재양자화 반올림과 시드	12
5.6 정수 구현 세부의 비용	13
5.7 학습된 반올림은 정수 실행에서도 남는가	14
5.8 외부 대조: TensorFlow Lite와 Arm Vela	15
6 논의	18
6.1 타당성 위협	18
6.2 측정이 찾아낸 구현 결함	18

7 결론	20
7.1 요약	20
7.2 향후 과제	20
A 정수 데이터패스 명세	22
B 재현 방법	23
참고문헌	25
Abstract	26

그림 차례

3.1	연산자별 국소 불일치 분포	5
5.1	노드별 코드 불일치	10
5.2	반올림 방식 대비 top-1 변화	12

표 차례

3.1	연산자별 국소 불일치	4
5.1	CIFAR-10 기준선	8
5.2	모의 양자화 대 정수 실행, 전체 테스트셋	9
5.3	LayerNorm 정수 에뮬레이션	10
5.4	밀집 출력 과제: ESPCN $\times 2$, 두 깊이	11
5.5	반올림 방식의 손해, 시드 3개	12
5.6	재양자화 구현 세부의 비용	13
5.7	재양자화 축, 시드 3개	13
5.8	AdaRound: 모의 경로와 정수 경로	14
5.9	TensorFlow Lite 참조 커널 교차 검증	15
5.10	Arm Vela(Ethos-U55-256) 대조	16
5.11	Vela 격차의 연산자별 분해	16

제 1장

서론

INT8 가속기에 신경망을 올리는 일은 서로 다른 두 프로그램이 결정한다. 질문에 답하는 쪽은 양자화-역양자화 (“모의 양자화”) 노드가 삽입된 부동소수점 그래프이고, 그 정확도가 곧 양자화 정확도로 보고된다 [1, 2]. 실제로 출하되는 쪽은 정수 프로그램이다. int8 가중치, int32 누산기, 고정소수점 곱셈-시프트 재양자화로 이루어진다 [3]. 현업은 둘을 바꿔 써도 되는 것으로 다루고, 정확도에 관한 한 대체로 그렇다 [4].

그러나 시뮬레이터에 요구되는 것이 정확도뿐인 것은 아니다. 실리콘 브링업과 컴파일러 검증에는 장치가 내놓아야 할 정수 텐서인 골든 벡터가 필요하다. 양자화 벤치마크는 배포 조건과 정확도의 관계를 다루고 [5], TensorFlow Lite Quantization Debugger는 계층별 오차와 전체 모델의 오차 전파를 분석하는 기능을 제공한다 [13]. 따라서 계층별 비교 자체를 새로운 방법으로 주장하지 않는다. 이 원고는 같은 양자화 파라미터를 쓰는 모의 양자화와 명시적 정수 실행을 코드 단위로 대조하고, 품질 지표와 코드 일치율이 다른 양상을 보이는지 측정하는데 초점을 둔다.

이 논문의 기여는 넷이다. 첫째, 모의 양자화와 정수 실행의 차이를 연산자 단위로 분해하는 측정 방법을 제시한다. 교사 강제로 분리한 국소 불일치는 연산자 자신의 정수 연산이 만드는 뭉을, 전파 불일치는 종단간 실행에서 관측되는 차이를 나타낸다. 둘째, 신경망 7종과 양자화 방식 2종의 품질·출력 코드는 전체 테스트셋에서 평가하고, 노드별 분해는 고정 표본 배치에서 수행한다. 품질 차이에는 대응 표준오차를 붙이고, 코드 불일치율은 별도로 보고한다. 셋째, 가장 자연스러운 수리 방법 — 가장 큰 국소 원천 하나에서 시뮬레이터가 장치의 정수 연산을 쓰게 하는 것 — 을 직접 시험하여, 국소 격차는 완전히 닫히지만 전파 격차는 거의 닫히지 않음을 보인다. 넷째, 정확도 대리지표가 볼 수 없는 재양자화 구현 선택을 시드를 바꿔 가며 정량화한다.

측정이 성립하려면 기준이 되는 정수 엔진 자체가 의심스럽지 않아야 한다. 이 논문의 엔진은 서로 독립인 두 구현(NumPy와 C++)이 시험 입력의 중간 텐서에서 일치할 것을 테스트가 강제하고, 독립 실행형 C++ 경로가 직렬화된 파일로부터 같은 코드를 재현하며, 무엇보다 자기 자신이 아니라 TensorFlow Lite 참조 커널이라는 외부 오라클에 대해 검증된다(5.7절). 5장의 모든 수치는 이 엔진 위에서 나온 것이다.

이 논문의 구성은 다음과 같다. 2장은 INT8 정수 데이터패스와 모의 양자화의 관계, 그리고 관련 연구를 정리한다. 3장은 연산자별 불일치를 먼저 분류해 문제의 모양을 드러낸다. 4장은 측정 방법을, 5장은 실험 결과를, 6장은 타당성 위협과 측정이 찾아낸 구현 결함을 다루고, 7장에서 정리한다.

제 2장

배경

2.1 INT8 정수 데이터패스

정수 추론의 한 계층은 세 단계로 이루어진다. int8 입력 코드와 int8 가중치를 곱해 int32 누산기에 모으고, int32 바이어스를 더하고, 결과를 다음 계층의 int8 격자로 되돌리는 재양자화를 거친다. 재양자화는 실수 배율을 Q31 고정소수점 승수 q 와 시프트 n 으로 근사한다.

$$M = \frac{s_{in} s_w}{s_{out}} \approx q \cdot 2^{-31} \cdot 2^{-n}.$$

계산은 gemmlowp/TensorFlow Lite의 MultiplyByQuantizedMultiplier [3]를 따른다. 이 함수는 포화 반올림 배증 상위곱과 2의 거듭제곱 반올림 나눗셈의 합성이며, 둘 다 정확히 정의된 정수 연산이다 (부록 A). 곧 “INT8”이라는 말이 가리키는 산술은 유일하지 않고, 승수 폭·누산기 폭·반올림 방식이 모두 구현이 고르는 값이다.

활성함수는 데이터패스에 따라 두 갈래로 갈린다. ReLU 계열은 재양자화의 clamp 경계에 흡수되므로 별도의 연산이 필요 없고, 이 구현에서 SiLU·GELU 같은 활성화함수는 256개 항목의 int8 룩업 테이블로 구현한다. 후자는 입력 쪽에 양자화기가 하나 더 붙는다는 뜻이고, 이 차이는 그래프의 양자화 지점 배치를 바꾼다. 평균 풀링은 입력 스케일을 그대로 유지한 채 정수 누적과 나눗셈으로 처리한다.

2.2 모의 양자화와 정수 실행

모의 양자화 그래프는 텐서를 양자화 격자에 맞춰 반올림하고 자른 뒤 다시 부동소수점으로 돌아온다. 곧 $\hat{x} = s(\text{clip}(\text{round}(x/s) + z, q_{\min}, q_{\max}) - z)$ 이고, 이후의 모든 연산은 여전히 float32로 돈다. 정수 프로그램은 대신 int8 코드를 끝까지 유지하고 위의 곱셈-시프트로 다시 스케일한다. 두 프로그램은 의도에서는 일치한다. 같은 스케일, 같은 영점, 같은 가중치를 쓴다. 그러나 연산의 순서와 정밀도가 다르므로 최하위 비트마다 어긋날 자유를 갖는다.

이 자유가 실제로 얼마나 쓰이는지가 이 논문의 주제다. 구체적으로 두 가지를 구별해야 한다. 한 연산자의 정수 커널이 같은 입력을 받고도 시뮬레이터와 다른 출력을 내는 몫과, 앞에서 생긴 차이가 뒤로 전달되며 커지는 몫이다. 4.3절은 이 둘을 각각 국소 불일치와 전파 불일치로 정의한다.

2.3 관련 연구

MQBench [5]와 양자화 백서들 [4, 2]은 양자화 방식과 배포 백엔드가 정확도에 미치는 영향을 다룬다. TensorFlow Lite Quantization Debugger의 계층별 검증은 동일한 출처의 입력을 두 계층에 주고 차이를 비교하며, 전체 모델 검증은 두 실행 경로를 분리해 오차 전파를 관찰한다. 직접 비교 지표에서는 부동소수점 값, 정수 값, 스케일과 영점에도 접근할 수 있다 [13]. 본 원고는 이와 관련된 진단 관점을 사용하되, 동일 파라미터의 모의 양자화와 정수 엔진 사이의 코드 불일치, LayerNorm 개입, 반올림 선택의 비용을 함께 측정한다. 새 정수 커널이나 계층별 비교 개념의 최초 제안이 아니라, 명시한 실행 조건에 대한 측정 연구다.

정수만 쓰는 트랜스포머 추론은 정수 LayerNorm과 소프트맥스 커널을 이미 확립했다 [6, 7]. 그 연산자에 대한 이 논문의 기여는 커널이 아니라, 측정한 차이의 크기와 LayerNorm 단독 교체의 한계다(5.3절). 누산기 폭은 학습 시 제약으로 연구되어 왔다 [11]. 여기서 재는 축은 그 대신 재양자화 단계의 반올림, 즉 순수한 구현 선택이다(5.5절). 비용 모델 쪽에서는 SCALE-Sim [12]과 같은 사이클 정확 시뮬레이터가 표준이며, 5.7절에서 이 논문의 분석 모델을 그 도구 및 Arm Vela에 대조한다.

표 3.1: 연산자별 국소(교사 강제) 불일치. 신경망 7종 × 양자화 방식 2종의 모든 노드를 연산자 종류로 모은 것이다. 국소 평균은 노드별 비율의 단순평균이다. “불일치 노드”는 국소 불일치가 0보다 큰 노드의 수, “국소 최대 $|\Delta|$ ”는 그 노드들에서 관측된 코드 차이의 최댓값(LSB), “전파 최대”는 같은 노드에서 종단간 실행이 만든 불일치의 최댓값이다.

연산자	노드	불일치 노드	국소 평균(%)	국소 범위(%)	국소 최대 $ \Delta $	전파 최대(%)
LayerNorm	26	26	24.766	23.637–26.226	1	82.8
평균 풀링	12	12	0.989	0.100–2.697	1	55.9
소프트맥스	12	12	0.500	0.276–0.798	1	26.7
행렬곱	24	24	0.273	0.076–0.425	1	75.6
합성곱	286	286	0.163	0.020–0.928	1	73.2
선형	84	84	0.139	0.031–0.960	1	80.8
잔차 덧셈	106	5	<0.001	0.000–0.001	1	75.6
LUT 활성화함수	50	0	0	0	0	39.5
연결(concat)	6	0	0	0	0	11.3

제 3장

동기: 격차는 어디에 있는가

모의 양자화가 정수 실행을 얼마나 못 맞히는지 묻기 전에, 못 맞히는 곳이 어디인지를 먼저 보아야 한다. “INT8 실행이 부정확하다”는 말은 설계 지침이 되지 못한다. 어떤 연산자가 얼마나 어긋나는지를 알아야 무엇을 고칠지, 고쳐서 될 일인지를 판단할 수 있다.

3.1 연산자별 불일치 분류

표 3.1와 그림 3.1는 이 논문이 다루는 신경망 7종 전부에 대해, 각 노드의 정수 커널에 시뮬레이터의 입력 코드를 먹이고(교사 강제) 그 노드의 출력만 비교한 결과를 연산자 종류별로 모은 것이다. 측정 절차는 4.3절에 있다. 국소 불일치는 고정 표본 배치에서 측정했다. 표의 평균은 노드별 불일치율의 단순평균이며, 텐서 원소 수로 가중한 평균이 아니다.

노드별 국소 불일치율의 단순평균은 LayerNorm이 24.77%로 가장 높다. 다른 연산자 종류의 평균은 모두 1% 미만이며, 평균 풀링은 0.989%로 두 번째다. 따라서 LayerNorm은 다음으로 큰 종류 평균의 약 25배다. 개별 노드까지 모두 1% 미만인 것은 아니며, 평균 풀링의 노드별 범위는 0.100–2.697%다. 합성곱 0.16%, 선형 0.14%, 행렬곱 0.27%, 소프트맥스 0.50%는 각 종류의 평균값이다.

크기보다 중요한 것은 어긋남의 종류가 하나뿐이라는 사실이다. 표 3.1의 “국소 최대 $|\Delta|$ ” 열은 불일치가 관측된 종류에서 1이고, 불일치가 없는 종류에서는 0이다. LayerNorm까지 포

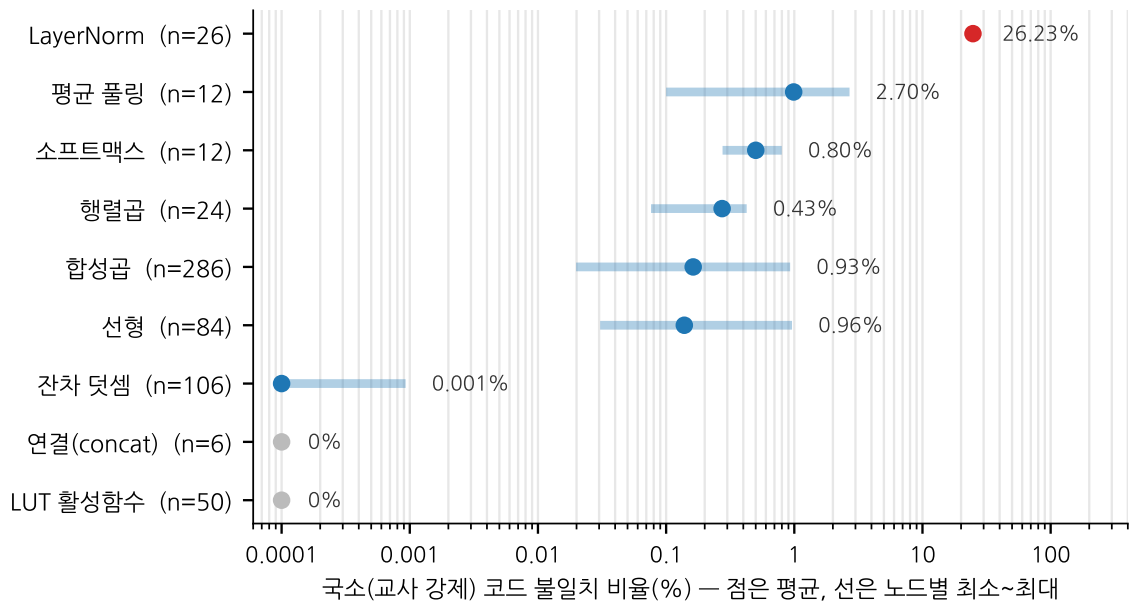


그림 3.1: 연산자별 국소 불일치 분포. 가로축은 로그 눈금이며, 국소 불일치가 정확히 0인 연산자(LUT 활성화함수, 연결)는 축 왼쪽 끝에 회색으로 붙여 표시했다.

함해, 교사 강제로 켜 모든 불일치는 최하위 비트 ± 1 이다. 정수 커널이 크게 틀리는 경우는 관측되지 않았고, 다만 ± 1 이 얼마나 자주 일어나는지가 연산자마다 다를 뿐이다.

세 종류의 연산자는 정확히 0%다. int8 룩업 테이블 활성화함수 50개 전부, 연결(concat) 6개 전부, 그리고 reshape·transpose 같은 구조 연산이다. 룩업 테이블은 입력 코드에서 출력 코드로 가는 결정적 사상이므로 시뮬레이터와 엔진이 같은 표를 쓰는 조건에서 국소 일치 가능성이 가능하다. 구조 연산은 코드의 위치를 바꾸며, 연결은 입력 스케일을 맞추는 재양자화가 포함될 수 있다. 여기서 관측된 연결 노드의 국소 차이는 0이었다. 잔차 덧셈 100개 중 95개도 0이고, 나머지 5개는 원소 100만 개당 몇 개 수준 (0.00005–0.00093%)에서만 어긋난다.

3.2 무엇이 문제인가

이 분류가 곧바로 말해 주는 것이 둘 있다.

첫째, 고칠 대상이 뚜렷하다. 트랜스포머를 INT8로 옮기는 실무자에게 “LayerNorm을 가장 먼저 점검한다”는 우선순위를 이 표에서 제안할 수 있다. 다만 이는 평가한 ViT와 양자화 조건에 대한 관찰이며, 모든 트랜스포머의 우선순위로 일반화하지 않는다.

둘째, 고쳐도 될지는 이 표만으로 알 수 없다. 국소 불일치는 각 연산자를 고립시켜 켜 값이고, 같은 표의 “전파 최대” 열은 전혀 다른 그림을 보여 준다. 국소 불일치가 정확히 0인 LUT 활성화함수의 전파 불일치가 39.5%까지 올라가고, 잔차 덧셈은 75.6%까지 간다. 국소 불일치가 작거나 0이어도 상류 차이와 이후 연산의 상호작용 때문에 전파 불일치가 클 수 있다. 그러므로 가장 큰 국소 원천을 없애면 전파 격차가 그만큼 줄어드는지는 따로 시험해야 한다. 5.3절이 그 시험이다.

제 4장

방법

4.1 그래프 IR과 정수 그래프 export

`torch.fx`로 추적한 그래프에서 BatchNorm을 합성곱에 접고 `shape`를 전파해 정적 그래프를 만든다. 지원 연산자는 합성곱(깊이별 포함), 선형, 덧셈, 곱셈, 연결, 행렬곱, 소프트맥스, LayerNorm, 풀링, 활성화함수, transpose, reshape, flatten이고, 그 밖의 노드를 만나면 조용히 근사하지 않고 즉시 실패시킨다. 측정 도구에서는 침묵하는 폴백이 가장 위험한 동작이기 때문이다.

캘리브레이션을 마친 모의 양자화 그래프는 정수 그래프로 export된다. 가중치는 int8, 바이어스는 int32, 그리고 출력 채널별 Q31 승수와 시프트가 붙는다. ReLU 계열 활성화함수는 이 단계에서 재양자화의 clamp로 흡수되고, LUT 활성화함수는 256항목 표로 구워진다.

4.2 정수 엔진과 외부 검증

실행은 2.1절의 gemmlowp/TensorFlow Lite 의미론을 따른다. 엔진은 세 겹으로 검증된다.

내부 일치. 서로 독립으로 작성한 NumPy 구현과 C++ 구현이 모든 중간 텐서에서 일치할 것을 테스트 스위트가 시험 입력에서 확인한다. 서로 다른 구현의 오류를 찾는 데 도움이 되지만, 두 구현이 같은 가정을 잘못 공유한 경우는 이 비교만으로 배제할 수 없다.

직렬화 왕복. 정수 그래프를 파일로 내보내고, 파이썬을 쓰지 않는 독립 C++ 실행기가 그 파일만으로 같은 코드를 재현한다. 파이썬 런타임에 숨은 상태가 없음을 보인다.

외부 오라클. 가장 중요한 겹이다. TensorFlow Lite의 완전 정수 모델을 가져와 그 스케일·영점·가중치·바이어스로 정수 그래프를 세우고, TensorFlow Lite 참조 커널과 출력을 비교한다. 결과는 5.7절에 있다. 자기 자신에 대한 일치는 구현이 일관된다는 것만 보이지만, 외부 런타임과의 비트 일치는 이 논문의 시험 모델에 포함된 합성곱·평균 풀링·완전 연결 계층이 지정한 참조 경로와 일치함을 보인다. 깊이별 합성곱·덧셈·소프트맥스·LayerNorm 전체에 대한 외부 검증이나 실제 NPU 실측을 뜻하지 않는다.

4.3 국소 불일치와 전파 불일치

한 장의 이미지에 대해, 모의 양자화 그래프가 노드 n 에서 만드는 코드를 f_n (양자화기 출력을 정수 코드로 환산한 것), 정수 엔진을 종단간으로 돌렸을 때 같은 노드에서 나오는 코드를 i_n 이라 하자. 노드 n 의 전파 불일치는 $f_n \neq i_n$ 인 원소의 비율이다.

국소 불일치는 노드 n 의 정수 커널에 입력으로 f 의 코드, 곧 시뮬레이터가 그 자리에서 내놓은 코드를 먹이고(교사 강제) 그 노드의 출력만 비교한 것이다. 상류에서 온 오차가 제거

되므로, 남는 차이는 그 연산자 자신의 정수 산술이 만든 것이다.

두 값은 서로를 보완한다. 국소 불일치만 보면 어떤 커널이 문제인지 알 수 있지만 그것이 결과에 얼마나 기여하는지는 알 수 없고, 전파 불일치만 보면 결과는 알 수 있지만 원인을 지목할 수 없다. 3장의 표에서 두 열이 전혀 다른 순위를 보인 것이 그 때문이다.

4.4 LayerNorm 정수 에뮬레이션

3장의 분류는 LayerNorm을 지목한다. 진단이 맞는지 확인하는 가장 직접적인 방법은 원인을 없애 보는 것이다. `emulate_integer_layernorm`은 캘리브레이션된 모의 양자화 그 래프의 `nn.LayerNorm`을, 입력 코드 위에서 정수 엔진과 같은 함수(int64 합, 정확한 정수 제곱근, Q15 정규화, 채널별 Q31 재양자화)를 수행하는 모듈로 바꾼다. 구성상 비트 단위까지 같고, 내보내는 정수 프로그램은 바뀌지 않는다. 실험은 후자를 단언(assert)한다. 곧 두 조건 사이에서 달라지는 것은 시뮬레이터뿐이다.

4.5 통계 처리

정확도 차이는 대응 표본으로 다룬다. 같은 이미지를 세 번(부동소수점, 모의 양자화, 정수) 평가하고, 이미지별 정답 여부 차이의 평균을 표준오차 $s/\sqrt{n}$ 및 95% Wald 구간과 함께 보고한다. 동일 이미지에서 얻은 결과의 대응 관계를 반영하기 위한 것이다. 신뢰구간이 0을 포함하는 것과 사전에 정한 허용 오차 내의 동등성을 입증하는 것은 다르다. 이 연구는 동등성 검정을 수행하지 않았다.

초해상처럼 이미지당 연속값(PSNR)으로 채점하는 과제에는 같은 방식의 연속형 대응 차이를 쓴다. 이미지별 차이의 평균과 그 표준오차, 95% 구간을 보고한다.

표 5.1: CIFAR-10 기준선. 사이클과 배열 활용률은 가상 NPU 비용 모델(tiny-1tops 스펙)의 값이므로 측정값이 아니라 추정값이다. lint 점수는 NPU 준비도 정적 분석의 종합 점수다.

모델	파라미터	MACs	float32 top-1(%)	사이클	배열 활용률(%)	lint 점수
ResNet-20 ReLU	272,474	40.8M	89.59	87.7k	45.4	92.9
ResNet-20 SiLU	272,474	40.8M	90.34	90.7k	44.0	75.3
MobileNetV2-0.5	700,490	28.0M	90.23	200.5k	12.1	88.2
ViT-128/6	810,890	57.0M	80.98	164.6k	33.8	75.3
Inception-32	423,066	54.0M	89.59	99.1k	53.2	97.0

제 5장

성능 평가

5.1 실험 설정

CIFAR-10 신경망 5종을 학습해 기준선으로 삼는다(표 5.1). ReLU를 쓰는 ResNet-20과 SiLU를 쓰는 ResNet-20, MobileNetV2-0.5(깊이별 합성곱), 분기를 연결하는 Inception 계열 CNN, 그리고 ViT-128/6(어텐션 블록 6개, 어텐션 행렬곱 노드 12개, LayerNorm 13개)이다. 여기에 Imagenette를 128×128 로 쓰는 신경망 2종을 더한다. ResNet-20과, 5.4절의 초해상 신경망이다.

학습 50,000장은 고정 시드로 학습 45,000장과 검증 5,000장(클래스별 500장, 층화)으로 한번 나눈다. 체크포인트 선택은 검증 분할로만 하고 test 10,000장은 선택된 체크포인트에 대해 마지막에 한 번 평가한다. 캘리브레이션 이미지도 학습 분할에서만 뽑는다. 이는 체크포인트 선택 절차에 대한 설명이며, 선택된 모델은 여러 양자화·개입 조건의 테스트 평가에 반복 사용된다.

학습 후 양자화 방식은 둘이다. 채널별 가중치에 비대칭 활성화를 쓰는 방식(PC)과 텐서별 가중치를 쓰는 방식(PT)이며, 둘 다 학습 이미지 512장으로 보정했다. 원래 학습 기록의 기준 환경은 CPU 4코어다. 평가 스크립트의 스레드 수는 설정에 따라 다르므로 재실행 시 실행 환경과 스레드 설정을 함께 기록해야 한다.

정확도 열은 따로 적지 않으면 전체 테스트셋 기준이다(CIFAR-10 10,000장, Imagenette 3,925장). 노드별 분해는 고정된 250장 배치를 쓰며, 입력이 64픽셀보다 큰 신경망은 125장(초해상은 64장)을 쓴다.

표 5.2: 모의 양자화 대 비트 단위로 일치하는 정수 실행, 전체 테스트셋. PC는 채널별 가중치, PT는 텐서별 가중치. Δ 는 같은 이미지에서 랜 (정수 - 모의) top-1이며 대응 표준오차를 함께 적었다. “코드”는 서로 다른 출력 로짓 코드의 비율, “이미지”는 코드가 하나라도 다른 이미지의 비율이다.

신경망	방식	float32	모의	정수	Δ (%p)	95% CI	일치(%)	코드(%)	이미지(%)
ResNet-20 ReLU	PC	89.59	89.54	89.55	$+0.01 \pm 0.08$	$[-0.14, +0.16]$	99.37	50.1	99
ResNet-20 ReLU	PT	89.59	89.61	89.67	$+0.06 \pm 0.08$	$[-0.11, +0.23]$	99.18	51.7	99
ResNet-20 SiLU	PC	90.34	90.35	90.26	-0.09 ± 0.09	$[-0.26, +0.08]$	99.06	61.6	100
ResNet-20 SiLU	PT	90.34	90.17	90.26	$+0.09 \pm 0.09$	$[-0.08, +0.26]$	99.07	62.9	100
MobileNetV2-0.5	PC	90.23	90.30	90.29	-0.01 ± 0.08	$[-0.16, +0.14]$	99.28	50.4	100
MobileNetV2-0.5	PT	90.23	90.23	90.27	$+0.04 \pm 0.07$	$[-0.10, +0.18]$	99.37	51.4	100
ViT-128/6	PC	80.98	80.87	80.97	$+0.10 \pm 0.13$	$[-0.15, +0.35]$	97.93	72.7	100
ViT-128/6	PT	80.98	80.77	80.97	$+0.20 \pm 0.12$	$[-0.04, +0.44]$	98.07	72.6	100
Inception-32	PC	89.59	89.55	89.56	$+0.01 \pm 0.05$	$[-0.09, +0.11]$	99.69	28.5	95
Inception-32	PT	89.59	89.52	89.47	-0.05 ± 0.06	$[-0.17, +0.07]$	99.52	31.1	96
ResNet-20 (Imagenette)	PC	83.97	84.05	83.97	-0.08 ± 0.10	$[-0.27, +0.12]$	99.52	35.7	95
ResNet-20 (Imagenette)	PT	83.97	84.00	84.05	$+0.05 \pm 0.11$	$[-0.17, +0.27]$	99.31	36.2	96

5.2 정확도와 텐서의 분리

표 5.2가 핵심 측정이다. 모의 대 정수 정확도 차이의 최댓값은 0.20 %p, $|\Delta|/SE$ 의 최댓값은 1.6이며, 12개 신뢰구간이 모두 0을 포함한다. 이는 각 비교에서 차이를 유의하게 검출하지 못했다는 뜻이며, 특정 허용 오차 안의 동등성을 입증한 것은 아니다. 이미지 단위 top-1 일치율은 97.9–99.7%다.

같은 행이 텐서에서는 어긋난다. 출력 로짓 코드의 28.5%(Inception-32)에서 72.7%(ViT-128/6)가 다르고, 이미지의 95–100%에서 코드가 최소 하나 다르다. 트랜스포머가 극단이기 하나 예외는 아니다. ResNet·MobileNet 계열은 약 50–63%, Inception은 약 28–31%, Imagenette 모델은 약 36%다.

그림 5.1은 이 결과를 노드 단위로 펼친 것이다. 국소 불일치는 3장에서 본 대로 작고 편이지만, 전파 불일치는 첫 합성곱에서 시작해 — 12개 행 전부에서 첫 격차가 그 노드다 — 노드에 따라 증가와 감소를 거치며 출력에서 28–73%에 이른다. 높은 top-1 일치율만으로는 시뮬레이터가 비트 정확한 검증 벡터를 공급한다고 판단할 수 없다.

5.3 LayerNorm 어블레이션

격차가 연산자별 오차의 합이라면 가장 큰 항을 없앴을 때 그만큼 줄어야 한다. 4.4절의 예물레이션으로 이를 직접 시험한다.

표 5.3이 결과다. LayerNorm의 국소 불일치는 24.77%에서 정확히 0.00%로 내려가고 로짓 절대차 평균은 약 23% 개선된다. 그러나 전파된 출력 코드 불일치는 72.7%에서 65.7%로만 움직인다 (텐서별은 72.6% → 66.0%). 정수 프로그램의 정확도는 바뀌지 않는다. 정수와 각 시뮬레이터의 차이 중 pc 교체 조건은 $+0.25 \pm 0.11$ %p로 약 2.2 표준오차다. 표의 각 차이는 정수 대비 값이지 두 시뮬레이터의 직접 대응 비교가 아니므로, 개입에 따른 정확도 예측의 개선을 이 표만으로 확정하지 않는다.

pc 조건에서 출력 코드 불일치는 약 7.0 %p, 상대적으로 약 9.6% 줄었다. 따라서 가장 큰

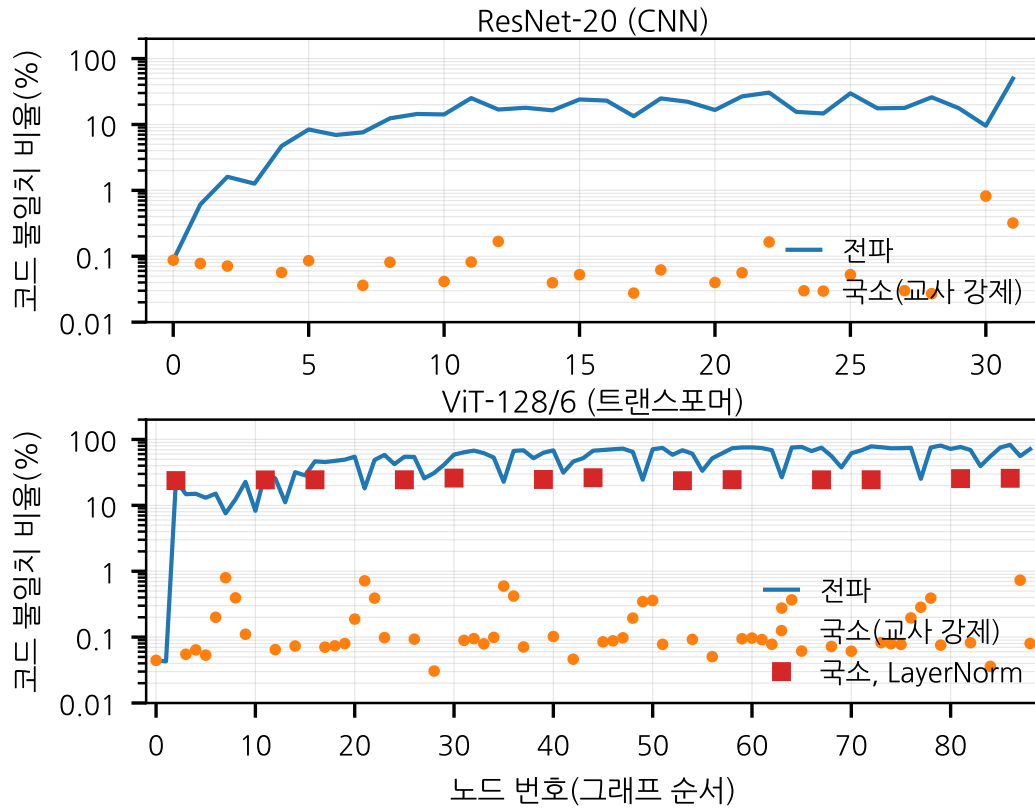


그림 5.1: 노드별 코드 불일치, PC 방식. 표시한 두 모델의 국소(교사 강제) 불일치는 LayerNorm에서 특히 크다. 전파 불일치는 첫 합성곱에서 시작해 출력의 50~73%까지 올라간다. 세로축은 로그 눈금이며, 국소 불일치가 0인 노드는 그리지 않았다.

표 5.3: 시뮬레이터의 LayerNorm을 정수에 충실하게 바꾼 결과(ViT-128/6, 10,000장). 각 방식의 두 행 사이에서 정수 프로그램은 바뀌지 않았고 시뮬레이터의 LayerNorm만 다르다. 로짓 절대차 평균은 PC에서 0.119 → 0.091, PT에서 0.118 → 0.093으로 줄어든다.

방식	LayerNorm	국소(%)	코드(%)	일치(%)	Δ (%p)
PC	float32	24.77	72.7	97.93	$+0.10 \pm 0.13$
PC	정수 (13개)	0.00	65.7	98.29	$+0.25 \pm 0.11$
PT	float32	24.76	72.6	98.07	$+0.20 \pm 0.12$
PT	정수 (13개)	0.00	66.0	98.60	$+0.13 \pm 0.10$

국소 원천인 LayerNorm만 맞추는 것으로는 출력의 비트 일치에 도달하지 못했다. 이 실험은 LayerNorm 단독 교체를 평가한 것이며, 다른 연산자들을 함께 또는 순차적으로 교체했을 때의 수렴 여부를 시험하지 않았다. 골든 벡터를 사용할 때는 해당 정수 의미론과 입력 조건에 대한 직접 검증이 필요하다.

표 5.4: 출력 텐서가 결과물인 과제: Imagenette에서의 ESPCN $\times 2$, 이미지 3,925장, 이미지당 출력값 49,152개를 같은 폭 구성의 두 깊이(합성곱 3개와 9개)에서 측정했다. PSNR은 $[0, 1]$ 범위의 dB이며 배포와 똑같이 출력을 clip한 뒤 이미지별로 재고 평균했다. $\max |\Delta c|$ 는 정수 실행과 시뮬레이터의 출력 픽셀 코드가 벌어진 최대 레벨 수다.

합성곱	방식	float32	모의	정수	Δ (10^{-3} dB)	코드(%)	$\max \Delta c $
3	PC	30.234	29.956	29.955	-0.64 ± 0.04	6.3	3
	PT	30.234	29.567	29.568	$+0.27 \pm 0.06$	18.2	3
9	PC	30.113	29.544	29.543	-0.51 ± 0.18	69.2	10
	PT	30.113	28.966	28.968	$+2.88 \pm 0.19$	73.0	10

5.4 출력 텐서가 결과물인 과제

5.2절과 5.3절의 신경망은 모두 분류기여서 출력 텐서와 답 사이에 $\arg\max$ 가 하나 서 있다. 코드가 수십 퍼센트 달라도 top-1은 거의 움직이지 않으므로, “텐서는 못 맞힌다”는 결론의 결과를 보여줄 수가 없다. 이 절은 $\arg\max$ 를 걷어낸다.

ESPCN [8]은 부분 픽셀 초해상 신경망이다. 합성곱 3개로 Imagenette를 $64 \rightarrow 128$ 로 올리며, 출력이 곧 그림 그 자체다. 파라미터 8,796개로 30 epoch(CPU 8.4분) 학습해 test PSNR 30.23 dB를 얻었고, 최근접 이웃 확대의 27.26 dB보다 2.97 dB 높다. 새 연산자 지원은 필요 없었다. 픽셀 서플을 `view/permute/reshape`로 쓰면 IR이 이미 가진 `reshape`와 `transpose`로 내려 간다.

이 신경망은 합성곱이 3개뿐이어서, 전파 불일치가 분류기들보다 작게 나오면 그것이 과제의 성질인지 깊이의 효과인지 구분할 수 없다. 그래서 같은 폭 구성을 유지한 채 본체에 잔차 블록 3개를 넣어 합성곱 9개짜리 변형을 따로 학습했다(파라미터 22,716개, test PSNR 30.11 dB). 이 변형은 더 좋은 신경망이 아니다. float PSNR은 오히려 0.12 dB 낮다. 같은 데이터·같은 폭·같은 학습 예산에서 연산자 수만 세 배로 늘린 대조군이며, 목적은 품질 개선이 아니라 깊이 하나만 바꿨을 때 전파 값이 어디로 가는지 재는 것이다.

표 5.4는 같은 그림을 반복하면서 깊이에 숫자를 붙인다. 합성곱 3개에서 양자화 비용은 0.279 dB(PC)와 0.667 dB(PT)인데, 시뮬레이터가 예측한 값과 정수 결과의 차이는 최대 약 6.4×10^{-4} dB다. 이 표본 크기에서는 분해되는 차이지만($|\Delta|/\text{SE} = 17.6$) 양자화 자체의 PSNR 변화보다 훨씬 작다. 그런데도 출력 픽셀 코드의 6.3%와 18.2%가 3,925장 전부에서 다르고, 최대 3레벨까지 벌어진단.

합성곱 9개에서는 양자화 비용이 0.569 dB와 1.147 dB로 커지고, 시뮬레이터와 정수 실행의 차이는 여전히 최대 약 2.9×10^{-3} dB에 머문다. 그동안 코드가 다른 비율은 69.2%와 73.0%로, 최대 편차는 10레벨로 오른다. 같은 과제·같은 데이터·같은 폭에서 연산자 수만 세 배로 늘린 결과이므로, 전파 값의 크기를 정하는 것은 과제가 아니라 깊이다. 합성곱 3개에서의 6.3%는 밀집 출력 과제의 성질이 아니라 그 그래프의 성질이었고, 9개는 분류기들이 차지하는 28–73% 구간 안으로 곧장 들어온다.

연산자별 구조는 두 깊이 모두 CNN과 같다. 합성곱의 국소 불일치는 0.06–0.40%이고 모두 최하위 비트 ± 1 이며, 첫 분기점도 첫 합성곱이다. 국소는 깊이에 거의 반응하지 않고 전파만 한 자릿수 움직인다는 점이 3.1절의 누적 구조를 그대로 보여 준다. 정확도 대리지표가 아니라

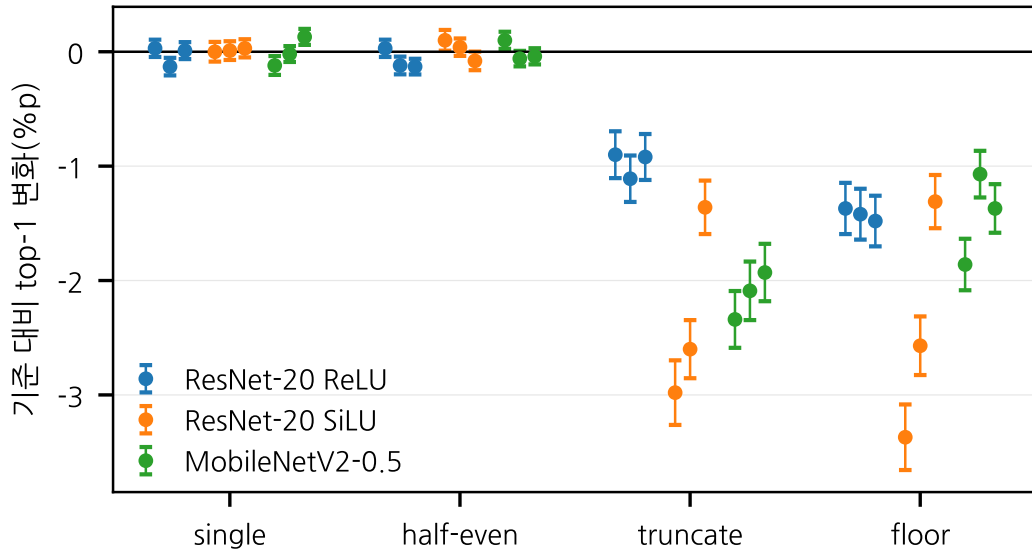


그림 5.2: 기준 재양자화 반올림 대비 top-1 변화, 전체 테스트셋, 신경망당 독립적으로 학습한 체크포인트 3개(오차 막대는 대응 표준오차).

표 5.5: 기준 구현 대비 반올림 방식의 손해, 시드 3개의 평균 ± 표준편차(%p).

반올림	ResNet-20 ReLU	ResNet-20 SiLU	MobileNetV2-0.5
single	-0.03 ± 0.09	+0.01 ± 0.02	-0.00 ± 0.13
half_even	-0.07 ± 0.09	+0.02 ± 0.09	+0.00 ± 0.09
truncate	-0.98 ± 0.12	-2.31 ± 0.85	-2.12 ± 0.21
floor	-1.42 ± 0.06	-2.42 ± 1.04	-1.43 ± 0.40

PSNR을 써도, 그리고 깊이를 바꿔도, 평균 품질의 미세한 차이와 코드 불일치는 계속 함께 관측된다.

5.5 재양자화 반올림과 시드

재양자화의 곱셈-시프트는 반올림 단계로 끝나는데, 구현에 따라 이를 단순 산술 시프트로 바꾸고 싶은 유혹이 있다. 정확도 대리지표는 이 선택에 값을 매기지 못한다. 그림 5.2과 표 5.5은 그 선택을 기준 구현과 비교해 전체 테스트셋에서, 신경망당 독립적으로 학습한 체크포인트 3개로 측정한 것이다.

결과는 둘이다. 첫째, 부호와 유의성은 시드에 강건하다. truncate와 floor는 18개 (모델, 시드, 방식) 조합 전부에서 기준보다 나빴고, 그 폭은 0.90–3.37 %p, $|\Delta|/SE$ 는 4.4에서 11.8 사이였다. 반면 단일 반올림과 짝수 반올림 경로는 잡음 범위를 벗어나지 않는다 ($|\Delta| \leq 0.13$ %p). 둘째, 크기는 옮겨 가지 않는다. ResNet-20 SiLU에서 truncate의 손해는 시드 3개에 걸쳐 1.36에서 2.98 %p까지 벌어져 시드 표준편차가 평균 손해 크기의 약 37%에 이르는 반면, ResNet-20 ReLU의 표에 표시된 비율은 약 12%다. 두 편향 방식 중 어느 쪽이 더 나쁜지도 연산의 성질이 아니라 신경망의 성질이다. ResNet-20 ReLU는 세 시드 모두에서 floor가, MobileNetV2는 세 시드 모두에서 truncate가 더 많이 잃는다. 반올림 방식의

표 5.6: 재양자화 구현 세부의 정확도 비용. 기준 설정 대비 정수 top-1 변화(%p), 이미지 2,000장, npu-default 스킴. 설정 이름의 m은 승수 비트, acc는 누산기 비트, b는 바이어스 비트다.

측	설정	ResNet-20 ReLU	ResNet-20 SiLU	MobileNetV2-0.5
곱셈기 비트	tfllite-m31-acc32-b32	기준	기준	기준
	tfllite-m15-acc32-b32	+0.10	+0.00	-0.25
	tfllite-m7-acc32-b32	+0.25	+0.00	+0.00
	tfllite-m3-acc32-b32	-8.20	-2.00	-1.55
누산기 비트	tfllite-m31-acc24-b32	+0.00	+0.00	+0.00
	tfllite-m31-acc20-b32	+0.00	+0.00	+0.00
	tfllite-m31-acc16-b32	-73.30	-31.55	-73.80
바이어스 비트	tfllite-m31-acc32-b16	-10.55	-0.80	-0.15
	tfllite-m31-acc32-b12	-76.65	-71.20	-73.45

표 5.7: 재양자화 구현 세부의 정확도 비용을 신경망당 체크포인트 3개로 다시 켜 것. 기준 설정 대비 정수 top-1 변화의 시드 평균 $\pm$ 시드 간 표준편차(%p), test 분할 전체 10,000장, npu-default 스킴. $\pm$ 뒤의 값은 쌍 표준오차가 아니라 시드 간 표준편차다.

설정	ResNet-20 ReLU	ResNet-20 SiLU	MobileNetV2-0.5
m15-acc32-b32	+0.02 $\pm$ 0.05	-0.01 $\pm$ 0.09	-0.05 $\pm$ 0.10
m7-acc32-b32	-0.05 $\pm$ 0.10	+0.08 $\pm$ 0.04	-0.01 $\pm$ 0.04
m31-acc24-b32	+0.00 $\pm$ 0.00	+0.00 $\pm$ 0.00	+0.00 $\pm$ 0.00
m31-acc20-b32	+0.00 $\pm$ 0.00	+0.00 $\pm$ 0.00	-0.04 $\pm$ 0.03
m31-acc16-b32	-59.95 $\pm$ 12.28	-32.15 $\pm$ 0.66	-74.42 $\pm$ 2.73
m31-acc32-b16	-6.25 $\pm$ 3.15	-0.44 $\pm$ 0.02	-0.62 $\pm$ 0.68
m31-acc32-b12	-78.33 $\pm$ 2.70	-70.19 $\pm$ 2.93	-72.36 $\pm$ 7.57

비용을 시드 하나의 수치로 인용해서는 안 된다.

5.6 정수 구현 세부의 비용

반올림은 재양자화가 고르는 여러 값 중 하나일 뿐이다. 표 5.6는 나머지 세 측 — 승수 폭, 누산기 폭, 바이어스 폭 — 을 기준 설정(Q31 승수, int32 누산기, int32 바이어스) 대비로 켜 것이다.

이 표에서 Q15·Q7 승수 설정의 관측 변화는 작지만, Q3 설정은 세 모델에서 각각 -8.20, -2.00, -1.55 %p였다. Q7부터 붕괴한다고 해석할 수 없다. 누산기 16비트와 바이어스 12비트 설정은 큰 하락을 보였다.

그런데 이 표의 숫자는 그대로 인용할 수 없다. 5.5절이 반올림 측에서 보인 것 — 효과의 부호는 체크포인트를 넘어 옮겨 가지만 크기는 옮겨 가지 않는다 — 이 여기에도 똑같이 걸리기 때문이다. 표 5.6는 모델당 체크포인트 하나와 2,000장으로 켜 것이다. 표 5.7는 같은 측을 신경망당 독립 학습 체크포인트 3개와 test 분할 전체(10,000장)에서 다시 켜 것이다.

결론은 두 문장이다. 등급은 시드를 넘어 유지되고, 크기는 유지되지 않는다.

표 5.8: AdaRound 대 최근접 반올림. “재구성 오차비”는 계층별 상대 재구성 오차의 AdaRound/최근접 비의 중앙값(작을수록 좋다), “방향 반전”은 최근접과 다른 쪽을 고른 가중치의 비율, 이득은 같은 이미지에서 최근접 반올림과 비교한 쌍 차이 $\pm$ 쌍 표준오차다. test 분할 전체 10,000장, AdaRound 1,500 iteration.

신경망	방식	재구성 오차비	방향 반전(%)	모의 이득(%p)	정수 이득(%p)
ResNet-20 ReLU	PC	0.46	4.9	$+0.06 \pm 0.09$	-0.02 ± 0.09
ResNet-20 ReLU	PT	0.41	6.6	-0.04 ± 0.10	-0.06 ± 0.10
ResNet-20 SiLU	PC	0.49	5.2	-0.06 ± 0.11	-0.10 ± 0.10
ResNet-20 SiLU	PT	0.45	6.8	$+0.13 \pm 0.11$	-0.06 ± 0.10
MobileNetV2-0.5	PC	0.58	4.0	-0.04 ± 0.10	$+0.06 \pm 0.10$
MobileNetV2-0.5	PT	0.55	5.8	-0.08 ± 0.10	-0.02 ± 0.10

무손실 등급은 완전히 무손실이다. int24와 int20 누산기는 (모델, 시드) 9조합 가운데 8 조합에서 기준과 10,000장 전부에 대해 같은 예측을 냈다(top-1 일치율 1.0000). 붕괴 등급은 모든 시드에서 붕괴한다. 그러나 그 크기는 인용할 수 없다. ResNet-20 ReLU의 int16 누산기는 시드에 따라 -72.82 , -58.68 , -48.36 %p이고, 같은 모델의 int16 바이어스는 -9.20 , -6.62 , -2.94 %p로 3배 넘게 벌어진다. 평균 효과가 0.10 %p 이상인 9개 조합에서 시드 간 표준편차를 평균 크기로 나눈 값의 최댓값은 109%다. 모델 사이의 순서도 뒤집힌다. int16 바이어스에서 MobileNetV2의 세 시드는 -0.31 , -1.40 , -0.16 %p이고 ResNet-20 SiLU는 -0.46 , -0.42 , -0.43 %p이므로, 시드를 하나만 고르면 두 모델의 상대 순위가 바뀐다.

따라서 표 5.6는 등급 판정용으로만 남기고, 크기를 말할 때는 표 5.7를 쓴다.

5.7 학습된 반올림은 정수 실행에서도 남는가

AdaRound [9]는 가중치를 최근접으로 반올림하는 대신 각 가중치의 반올림 방향을 그 계층의 출력 오차로 학습한다. 학습 후 양자화에서 표준으로 인용되는 방법인데, 보고되는 수치는 예외 없이 모의 양자화 그래프 위의 정확도다. 3장의 결론을 여기에 대면 자연스러운 질문이 하나 나온다. 시뮬레이터에서 얻은 이득이 실제로 나가는 정수 프로그램에도 남는가?

이 저장소는 그 질문을 던질 수 있게 만들어져 있다. 학습된 반올림은 양자화 계층의 w_round 버퍼에 들어가고 int_weight()가 같은 버퍼를 읽으므로, export된 정수 그래프는 최적화된 바로 그 반올림을 실행한다(계층별로 node.w_int와 module.int_weight()의 일치를 시험이 확인한다).

결과는 음성(null)이고, 음성인 방식이 정확히 흥미롭다. AdaRound가 최적화하는 목적함수는 모든 계층에서 좋아진다. 계층 194개 전부에서 그 계층의 재구성 오차가 줄었고 중앙값으로 절반 아래다 (비 0.41–0.58). 가중치의 4.0–6.8%가 최근접과 다른 방향으로 간다. 구현은 논문대로 동작한다. 그런데 top-1은 어느 경로에서도 움직이지 않는다. 6개 (신경망, 방식) 조합 전부에서 $|\Delta|/SE \leq 1.2$ 다.

이것은 AdaRound에 대한 반박이 아니다. AdaRound가 보고하는 이득은 4비트 이하 가중치에서 나오는데, 이 장의 조건은 8비트 가중치이고 그 조건에서는 최근접 반올림 자체가 이미 사실상 무손실이다(5.2절). 회복할 여유가 없는 곳에서는 회복할 것이 없다. 이 논문이 덧붙일

표 5.9: TensorFlow Lite 참조 커널과의 교차 검증. 이미지 1,000장, 출력 원소 10,000개. tflite+fc-single은 합성곱·풀링이 이중 반올림, 완전 연결 계층이 단일 반올림을 쓰는 조합이다.

반올림 조합	엔진	불일치 이미지	불일치 원소	top-1 일치(%)	완전 일치
tflite+fc-single	numpy	0 / 1000	0 / 10000	100.0	예
tflite+fc-single	cpp	0 / 1000	0 / 10000	100.0	예
tflite	numpy	20 / 1000	20 / 10000	100.0	아니오
tflite	cpp	20 / 1000	20 / 10000	100.0	아니오
single	numpy	155 / 1000	337 / 10000	100.0	아니오
single	cpp	155 / 1000	337 / 10000	100.0	아니오
half_even	numpy	182 / 1000	364 / 10000	100.0	아니오
half_even	cpp	182 / 1000	364 / 10000	100.0	아니오
truncate	numpy	1000 / 1000	8203 / 10000	99.8	아니오
truncate	cpp	1000 / 1000	8203 / 10000	99.8	아니오
floor	numpy	1000 / 1000	7968 / 10000	99.8	아니오
floor	cpp	1000 / 1000	7968 / 10000	99.8	아니오

수 있는 것은 두 경로의 대조다. 시뮬레이터에서 +0.13 %p였던 조합(ResNet-20 SiLU, pr)이 정수 프로그램에서는 -0.06 %p다. 둘 다 잡음 크기이지만, 6조합 중 3조합에서만 부호가 같다. 잡음 크기의 시뮬레이터 이득은 부호조차 장치로 옮겨 가지 않는다. 8비트에서 0.1 %p짜리 학습 후 양자화 개선을 보고하려면 정수 실행에서 다시 재야 한다.

재지 않은 것도 적어 둔다. 4비트 이하 가중치, ImageNet 규모 신경망, 블록 단위 재구성 (BREQ [10])은 이 실험의 범위 밖이고, AdaRound가 이득을 보이는 조건이 바로 그쪽이다.

5.8 외부 대조: TensorFlow Lite와 Arm Vela

TensorFlow Lite 참조 커널

표 5.9는 4.2절의 외부 검증 결과다. TensorFlow Lite 완전 정수 모델의 스케일·영점·가중치·바이어스로 정수 그래프를 세우고 1,000장을 흘렸을 때, 합성곱과 풀링이 기존 방식의 이중 반올림을 쓰고 완전 연결 계층이 단일 반올림을 쓰는 조합에서 모든 텐서가 완전히 일치한다. NumPy 엔진과 C++ 엔진이 같은 값을 낸다.

이 기준이 얼마나 뻥뻥한지에 대한 눈금이 하나 있다. 같은 모델에서 TensorFlow Lite 자신의 최적화 텔리게이트(XNNPACK)와 참조 커널은 1,000장 중 155장, 출력 원소 10,000개 중 337개에서 서로 다른 답을 낸다. 곧 “TensorFlow Lite와 비트가 맞는다”는 말조차 어느 커널을 가리키는지 밝히지 않으면 성립하지 않는다. 표의 나머지 행은 반올림 방식을 바꿨을 때 일치가 어떻게 깨지는지를 보여 준다. truncate와 floor는 1,000장 전부에서 어긋난다.

비용 모델

비용 모델 쪽 대조는 두 가지다. SCALE-Sim v3(사이클 정확, weight-stationary)와는 신경망 4종 × 배열 크기 3종에서 합계 사이클이 0.08% 이내로 일치한다. Arm Vela(Ethos-U55-256)와는

표 5.10: Arm Vela(Ethos-U55-256)와의 대조. 같은 INT8 그래프에 대한 두 분석 추정기의 값이며, 비율은 npuloop/Vela다. 둘 다 추정값이고 실측이 아니다.

합성 망	Vela MACs	npuloop MACs	Vela 사이클	npuloop 사이클	비율
conv-stack	3.06M	3.06M	47.8k	40.2k	0.840
dw-separable	0.43M	0.43M	22.4k	10.7k	0.477
pointwise-heavy	1.81M	1.79M	44.4k	12.7k	0.286
wide-late	2.77M	2.77M	61.5k	50.6k	0.822
deep-narrow	0.79M	0.79M	15.8k	11.3k	0.711

표 5.11: Vela와의 사이클 격차를 연산자 종류로 분해한 것. “격차 비중”은 $\sum(\text{Vela} - \text{npuloop})$ 가운데 그 종류가 차지하는 몫이다. 합계 비율 0.655는 거의 전적으로 첫 행에서 나온다.

연산자	계층	Vela 사이클	npuloop 사이클	npuloop/Vela	격차 비중(%)
pool	5	49,412	544	0.011	74.1
conv	20	132,384	118,241	0.893	21.4
dwconv	2	4,928	2,880	0.584	3.1
linear	5	4,640	3,736	0.805	1.4
합계	32	191,364	125,401	0.655	100.0

합성 망 5종에서 표 5.10와 같이 갈린다.

MAC 수는 사실상 같지만 합계 사이클은 0.29–0.84배로, npuloop 쪽이 일관되게 낙관적이다. 합계 비율 하나만 보면 “전체적으로 낙관적”이라는 말밖에 할 수 없다. 이 절의 나머지는 그 비율을 계층 단위로 분해해 낙관의 출처를 특정한다. 두 도구가 같은 계층 목록을 내놓는 32개 계층에 대해 사이클을 연산자 종류로 모으면 표 5.11가 된다.

격차는 고르게 퍼져 있지 않다. 평균 풀링 계층 5개가 전체 격차의 74.1%를 차지하며, 이 종류에서 npuloop의 값은 Vela의 0.011배, 즉 두 자릿수 과소평가다. 원인은 추정 오차가 아니라 서로 다른 낮춤(lowering)이다. Vela는 전역 평균 풀링을 depthwise 합성곱과 곱셈으로 낮추어 $H \times W \times C$ 개의 MAC을 계상하는 반면, npuloop의 비용 모델은 풀링에 MAC을 매기지 않고 벡터 패스 한 번으로 센다. 두 도구의 MAC 수가 매번 정확히 전역 평균 풀링 하나만큼 어긋나는 것도 같은 이유다. 풀링을 빼고 나면 합성곱의 비율은 0.893으로, 남는 낙관은 11%다.

그 11%에도 구조가 있다. 합성곱만 따로 모아 망별로 보면 conv-stack 0.964, wide-late 1.024, deep-narrow 0.895, dw-separable 0.671, pointwise-heavy 0.563이다. 일반 합성곱이 대부분인 망에서는 두 추정기가 4% 안에서 만나고, 1×1 합성곱과 깊이별 합성곱이 많아질수록 우리 쪽이 싸게 매긴다. 이 두 연산자는 MAC당 가중치 재사용이 적어 대역폭에 묶이는데, npuloop는 배열이 채워진다고 보는 반면 Vela는 그 대역폭 제약을 계상한다.

이 분해는 두 번째 원인 후보를 좁힐 뿐 확정하지 않는다. Vela는 컴파일된 스케줄(융합·타일링·cascading)을 계상하고 npu.estimate는 컴파일러 없이 계층을 하나씩 세므로, 남은 11% 가운데 비용 항의 몫과 스케줄의 몫을 가르려면 Vela의 스케줄 출력과 계층별로 대조해야 한다. 그것은 이 논문의 범위 밖이다. 또한 여기서 비용 모델을 Vela에 맞추어 보정하지는 않았다. 보정 대상이 실측 실리콘이 아니라 또 하나의 분석 추정기이므로, 맞춘 결과가 무엇에 대한

정확도인지 말할 수 없기 때문이다. 표의 사이클은 두 추정기의 값이며, 실제 하드웨어에서 측정한 성능이 아니다.

제 6장

논의

6.1 타당성 위협

외적 타당성. 신경망은 CIFAR-10 규모에 128×128 Imagenette 모델 셋을 더한 것이고, 가장 깊은 것이 연산자 32개다. 따라서 결론은 격차의 구조에 관한 것이지 ImageNet 규모 그래프에서의 정확한 크기에 관한 것이 아니다. 규모 축 가운데 직접 바꿔 본 것은 깊이 하나다(5.4절). 폭과 과제를 고정한 채 합성곱 3개를 9개로 늘렸더니 전파 불일치가 6.3%에서 69.2%로 한 자릿수 움직였다. 그러므로 이 논문이 보고하는 전파 비율은 측정한 그래프의 성질로 읽어야 하며 INT8이라는 수치 형식의 상수로 읽어서는 안 된다. 다만 깊이를 두 점에서만 켜므로 이 관계의 함수 형태는 알 수 없고, 더 깊은 그래프에서 포화하는지도 확인하지 않았다. 비용 모델은 가상 NPU 스펙 위에서 돌며, 실측 실리콘이 아니라 두 분석 도구에 대조했을 뿐이다(5.7절).

구성 타당성. 국소 불일치는 교사 강제로 재드로 연산자의 산술만 떼어 보고 상류 오차와의 상호작용은 보지 않는다. 그 상호작용을 담는 것은 전파 열이며, 3.2절에서 둘을 같은 문장에서 섞지 말아야 한다고 적은 이유가 그것이다. 또한 “코드 불일치”는 개수를 셀 뿐 크기를 재지 않는다. 모든 국소 불일치가 ± 1 LSB로 밝혀졌으므로(3.1절) 이 지표는 이 연구의 범위에서는 충분하지만, 더 큰 오차가 생기는 구현에서는 크기를 함께 보고해야 한다.

결론 타당성. 표 5.2의 정확도 비교 12개와 표 5.3의 4개는 다중비교 보정을 하지 않았다. 16개 전체에서 $|\Delta|/SE$ 의 최댓값은 2.2다. 시드 3개는 재양자화 축 전체(반올림, 승수 폭, 누산기 폭, 바이어스 폭)를 덮지만 나머지 측정은 신경망당 체크포인트 하나를 쓴다. 시드 3개는 크기가 흔들린다는 것을 보이기에 충분해도 그 분포를 특성화하기에는 적다. 이 체크포인트들을 만들 당시 학습기의 시드는 데이터 순서만 통제하고 가중치 초기화는 통제하지 못했으므로, 세 반복은 서로 다른 학습 반복이지만 같은 명령으로 비트 단위까지 재현되지는 않는다. 학습기는 이후 수정했다. 과거 표의 정확한 재측정에는 당시 원본 체크포인트와 평가 환경·보정 표본 정보가 필요하다.

내적 타당성. 외부 비트 일치 검증(5.7절)은 합성곱·풀링·완전 연결 계층을 덮고, 깊이별 합성곱·원소별 덧셈·소프트맥스는 덮지 않는다. 이 연산자들에 대해서는 내부 두 구현의 일치와 직렬화 왕복만 보장된다.

6.2 측정이 찾아낸 구현 결함

5.4절의 밀집 출력 과제를 추가하는 과정에서 측정 도구 자체의 결함이 하나 드러났다. 기록해 둘 가치가 있는 것은 그 결함이 새로운 종류의 입력을 넣기 전에는 보이지 않았기 때문이다.

처음 측정했을 때 초해상 신경망의 출력 불일치는 46.0%와 46.3%였고, 노드별로 펼쳐 보니 그 대부분이 첫 합성곱이 아니라 입력 양자화기에서 나왔다. 입력 코드의 7.16%가 시뮬레이터와 달랐다. 분류기에서는 한 번도 관측되지 않은 현상이다.

원인은 정밀도였다. 엔진의 `quantize_input`은 입력을 `float64`로 올린 뒤 나누고 반올림하는데, 시뮬레이터(PyTorch)는 `float32`로 나눈다. 둘 다 `round-half-to-even`이지만, 한쪽에서 정확히 동점인 값이 다른 쪽에서는 동점이 아니다. 초해상의 저해상 입력은 `uint8` 네 개의 박스 평균이므로 $1/1020$ 의 배수이고, $1/255$ 스케일에 대해 네 값 중 하나가 정확히 반올림 동점에 놓인다. 분류기 입력은 채널별 평균과 표준편차로 정규화되어 있으며, 평가한 분류 입력에서는 이 정밀도 변경으로 코드가 바뀌지 않았다. 이를 모든 분류 입력에서 결함이 발생할 수 없다는 뜻으로 일반화하지 않는다.

시뮬레이터와 같은 `float32`로 맞추자 입력 불일치는 0.00%가 되고 출력 불일치는 6.3%와 18.2%로 내려갔다. 기존 결과는 영향을 받지 않는다. 신경망 6종 × 양자화 방식 2종 × 전체 테스트셋에서 입력 코드 693,043,200개 중 **0개**가 바뀌므로 하류의 모든 텐서가 비트 단위로 동일하다. 회귀 테스트는 옛 경로가 실제로 어긋났음을 먼저 단언한 뒤 새 경로의 일치를 확인하므로 공허하게 통과하지 않는다.

이 사례의 교훈은 측정 도구에 국한되지 않는다. 입력 전처리가 값을 거친 격자 위에 올려 놓는 파이프라인 — 다운샘플, 양자화된 센서 출력, 정수 연산으로 만든 특징 — 에서는 입력 양자화기 자체가 첫 번째 분기점이 될 수 있다. 격차를 노드별로 펼쳐 보지 않았다면 이 7.16%는 “초해상은 원래 더 많이 어긋난다”로 오해되었을 것이다.

제 7장

결론

7.1 요약

평가한 분류 조건의 12개 비교에서는 관측 정확도 차이가 최대 0.20 %p였고 각 신뢰구간은 0을 포함했지만, 출력 코드의 28.5–72.7%는 달랐다. 초해상 조건에서도, 그리고 그 신경망의 깊이를 세 배로 늘린 조건에서도, 평균 PSNR의 작은 차이와 코드 불일치가 함께 관측됐다. 이는 품질의 유사성만으로 비트 일치를 보장할 수 없음을 보이며, 정확도 동등성을 입증한 결과는 아니다. 전파 불일치의 크기 자체는 그래프마다 다르며, 이 논문이 직접 바꿔 본 축에서는 깊이가 그것을 한 자릿수 움직였다.

연산자 종류별 노드 평균은 LayerNorm 이외에서 1% 미만이지만, 평균 풀링의 개별 노드 최대는 2.697%였다. 관측된 비영 국소 코드 차이는 모두 ± 1 LSB였다. LayerNorm만 정수로 에뮬레이션한 pc 조건에서 출력 코드 불일치는 약 9.6% 상대 감소했으나 0이 되지 않았다. 다른 연산자들의 누적 교체와 더 큰 모델에 대해서는 추가 실험이 필요하다.

실무적 함의는 둘이다. 골든 벡터가 필요한 팀은 외부 기준에 대해 검증된 정수 구현에서 그것을 만들어야 한다. 그리고 재양자화의 세부를 바꾸는 팀은 여러 시드에서 측정해야 한다. 그 효과의 크기는 — 부호와 달리 — 옮겨 가지 않기 때문이다.

7.2 향후 과제

규모. 가장 분명한 다음 단계는 ImageNet 규모 그래프다. 이 논문의 결론은 격차의 구조에 관한 것이지만, 깊이가 늘어날 때 전파 불일치가 어디서 포화하는지는 측정되지 않았다.

밀집 출력의 깊이. 5.4절은 합성곱 3개와 9개 두 점에서 깊이를 켜고, 전파 불일치가 6.3%에서 69.2%로 올라 분류기 수준(28–73%) 안으로 들어오는 것을 확인했다. 남은 것은 함수 형태다. 두 점으로는 이 상승이 어디서 포화하는지 알 수 없고, 분할·검출처럼 출력 구조가 다른 밀집 과제에서 mIoU나 mAP가 PSNR만큼 충실한지도 답하지 않았다.

크기의 분포. 5.6절이 재양자화 축 전체를 시드 3개로 다시 켜고, 크기가 흔들린다는 것은 확인됐다 (int16 누산기에서 -72.8부터 -48.4 %p까지). 확인되지 않은 것은 그 분포다. 시드 3개로는 흔들림이 체크포인트의 어떤 성질에서 오는지 — 초기화인지, 데이터 순서인지, 수렴한 극소점의 형태인지 — 를 가릴 수 없다.

비용 모델. 5.7절에서 Vela와의 격차를 연산자 종류까지 좁혔다. 전역 평균 풀링의 낮춤 방식이 격차의 74.1%이고, 나머지는 1×1 ·깊이별 합성곱의 대역폭 향이 후보다. 남은 일은 이를 Vela의 스케줄 결과와 계층별로 대조해 확정하고, 고친 비용 모델이 별도 평가 그래프에서도

유지되는지 보는 것이다. 다만 보정 대상이 실측 실리콘이 아니라 또 하나의 추정기인 한, 맞춘 결과를 “정확해졌다”고 부를 수는 없다.

제 A장

정수 데이터패스 명세

이 부록은 5장의 결과를 재현하는 데 필요한 정수 산술의 정의를 모은 것이다.

재양자화. 여기서는 $0 < M < 1$ 인 오른쪽 시프트 경로를 표시한다. 이 범위의 실수 배율 M 을 $q \in [2^{30}, 2^{31})$ 과 시프트 $n \geq 0$ 으로 분해해 $M \approx q \cdot 2^{-31} \cdot 2^{-n}$ 으로 근사한다. int32 누산값 a 에 대해

$$\text{requant}(a) = \text{RDBPOT}(\text{SRDHM}(a, q), n),$$

여기서 **SRDHM**은 포화 반올림 배증 상위곱, **RDBPOT**은 2의 거듭제곱 반올림 나눗셈이다. $M \geq 1$ 에 필요한 왼쪽 시프트 경로와 경계 처리까지 이 단순식이 나타내지는 않는다. 기준 구현은 두 단계 모두에서 반올림하며(**tfllite**), 비교 대상인 **single**은 한 번만, **half_even**은 짝수 반올림을, **truncate**와 **floor**는 반올림 없이 자른다.

활성함수. ReLU 계열은 재양자화 뒤 $[q_{\min}, q_{\max}]$ **clamp**에 흡수된다. SiLU, GELU 및 HardSwish는 입력 코드 256개를 출력 코드로 보내는 int8 룩업 테이블이며, 표는 **export** 시점에 구워진다.

LayerNorm. 입력 코드 d 에 대해 int64로 합과 제곱합을 구하고, 분산의 제곱근을 정확한 정수 제곱근(**isqrt**)으로 계산한 뒤 Q15로 정규화하고, 채널별 Q31 승수와 시프트로 재양자화한다. 4.4절의 에뮬레이션 모듈은 이 절차를 그대로 수행한다.

평균 풀링. 입력 스케일을 유지한 채 int32로 누적하고 창 크기로 반올림 나눗셈한다. 스케일이 바뀌지 않으므로 재양자화가 없다.

제 B장

재현 방법

저장소는 결과 JSON에서 표와 그림을 만드는 생성기를 제공한다. 기존 지속적 통합의 표 동기화 검사는 README와 실험 문서를 대상으로 하며, 논문용 표·PDF 검사는 별도로 추가·실행해야 한다. 표 재생성은 저장된 JSON과 표의 일관성을 검사하는 작업이지, 학습·평가를 독립 재실행하거나 본문의 모든 해석을 자동 검증하는 작업이 아니다. 본 수정판의 표는 해당 커밋의 생성된 TeX 파일을 유지했으며, 그림은 업로드된 원본 PDF의 벡터 그림을 재사용했다. 새 학습·평가 결과는 추가하지 않았다.

측정	스크립트
표 5.1 (기준선)	experiments/e1_baselines.py
표 3.1, 그림 3.1	e15_fidelity.py + e17_dense_output.py
표 5.2, 그림 5.1	experiments/e15_fidelity.py
표 5.3 (LayerNorm 어블레이션)	experiments/e16_ln_emulation.py
표 5.4 (밀집 출력)	experiments/e17_dense_output.py
표 5.5, 그림 5.2	experiments/e14_rounding_seeds.py
표 5.6 (구현 세부)	experiments/e7_requant_ablation.py
표 5.9 (TFLite 교차 검증)	experiments/e11_tflite_crosscheck.py
표 5.10 (Vela 대조)	experiments/e13_vela.py

표와 그림의 재생성은 다음 세 명령이다.

```
python paper/make_tables.py --lang ko
python paper/make_figs.py --lang ko
python paper/make_thesis.py
```

학습된 체크포인트는 저장소에 포함되지 않는다. 데이터셋 없이 돌려 볼 수 있도록 작은 체크포인트 2개와 CIFAR-10 표본이 examples/quickstart/에 들어 있고, 나머지는 python -m npuloop.zoo.train과 python -m npuloop.zoo.train_sr로 새로 학습할 수 있다. 단, 당시 초기화가 완전히 고정되지 않은 원본 체크포인트와 같은 가중치나 논문의 같은 소수점 결과를 복원한다는 보장은 없다. 정확한 재측정에는 원본 가중치·체크섬, 환경 버전, 데이터 분할 및 캘리브레이션 표본 정보가 필요하다. 원래 초해상 학습 기록은 CPU 4코어, 30 epoch, 8.4분이다.

참고문헌

- [1] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.
- [2] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” *arXiv:1806.08342*, 2018.
- [3] B. Jacob *et al.*, “gemmlowp: a small self-contained low-precision GEMM library,” 2017. [Online]. Available: <https://github.com/google/gemmlowp>
- [4] M. Nagel *et al.*, “A white paper on neural network quantization,” *arXiv:2106.08295*, 2021.
- [5] Y. Li *et al.*, “MQBench: Towards reproducible and deployable model quantization benchmark,” in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.
- [6] S. Kim *et al.*, “I-BERT: Integer-only BERT quantization,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 5506–5518.
- [7] Y. Lin *et al.*, “FQ-ViT: Post-training quantization for fully quantized vision transformer,” in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2022, pp. 1173–1179.
- [8] W. Shi *et al.*, “Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 1874–1883.
- [9] M. Nagel *et al.*, “Up or down? Adaptive rounding for post-training quantization,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 7197–7206.
- [10] Y. Li *et al.*, “BRECQ: Pushing the limit of post-training quantization by block reconstruction,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.
- [11] I. Colbert *et al.*, “A2Q: Accumulator-aware quantization with guaranteed overflow avoidance,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 16989–16998.
- [12] A. Samajdar *et al.*, “A systematic methodology for characterizing scalability of DNN accelerators using SCALE-Sim,” in *Proc. IEEE Int. Symp. Performance Analysis of Systems and Software (ISPASS)*, 2020, pp. 58–68.

- [13] Google, “Inspecting quantization errors with Quantization Debugger” and “QuantizationDebugOptions,” official LiteRT/TensorFlow Lite documentation, accessed September 16, 2026. [Online]. Available: https://ai.google.dev/edge/litert/conversion/tensorflow/quantization/quantization_debugger; <https://ai.google.dev/edge/api/tflite/python/tf/lite/experimental/QuantizationDebugOptions>

Abstract

Measuring the Fake-Quantization-to-Integer Gap, Operator by Operator

This manuscript compares fake-quantized floating-point execution with explicit integer execution using the same quantization parameters. For an external cross-check, the NumPy and C++ engines agree with TensorFlow Lite reference kernels on all compared tensors for 1,000 images of a test model covering convolution, mean pooling, and fully connected layers. This does not establish equivalence for all supported operators or for physical NPU hardware.

Across six classification networks and two schemes, the twelve observed top-1 differences are at most 0.20 percentage points in magnitude and all individual 95% confidence intervals contain zero; this is not an equivalence test. Nevertheless, 28.5–72.7% of the output codes differ. Quality and output-code metrics use complete test splits, while teacher-forced local comparisons use fixed sample batches.

The unweighted mean of node-local mismatch rates is below 1% for each operator class other than LayerNorm, but an individual pooling node reaches 2.697%. All observed nonzero local code differences are ± 1 LSB. Making LayerNorm alone integer-faithful removes its local mismatch (24.77% to 0.00%) yet reduces propagated output mismatch only from 72.7% to 65.7%. The experiment establishes the limitation of this intervention, not the impossibility of cumulative operator corrections.

A seventh, super-resolution network shows the same separation: the largest mean PSNR difference is approximately 6.4×10^{-4} dB while 6.3–18.2% of pixel codes differ. Removing requantization rounding in the truncate and floor modes costs 0.90–3.37 percentage points in all eighteen comparisons (three models, three training repeats, two rounding modes). The manuscript also documents an input-quantizer precision defect exposed by this task. Table regeneration from stored results is distinguished from independent experiment reproduction; the missing original checkpoints remain a limitation.

Keywords: Quantization, integer arithmetic, embedded inference, neural processing units, reproducibility